

Hvordan konfigurere Argo CD Application

Denne guiden vil hjelpe deg med å konfigurere Argo CD Application til å deploye kubernetes-manifester.

Introduksjon

Argo CD konfigureres av utviklere med [Application](#)-manifester. Denne guiden viser deg hvordan disse opprettes og konfigureres ved å skrive `Application`-manifester manuelt.

`Application`-manifester kan også genereres automatisk ved hjelp av [ApplicationSet](#). `ApplicationSet` er en mal som automatisk bygger opp `Application`-manifester basert på standard-konfigurasjon. Les mer om `ApplicationSet` i [Bruke ApplicationSet](#).

Det lønner seg å ha grunnleggende forståelse for `Application`-manifester når man benytter `ApplicationSet`.

Når burde jeg bruke `Application` og når burde jeg bruke `ApplicationSet`

Det anbefales å benytte `ApplicationSet` så langt man kan. Dersom man har spesielle behov som ikke dekkes av `ApplicationSet`-malen kan man skrive `Application`-manifestet manuelt.

Forutsetninger

- Argo CD er satt opp som en del av tenant-bestillingen. Se eventuelt [Endre namespace](#).
- Argo CD har tilgang til git-repo med manifester. Se eventuelt [Tilgangstyring](#).

Konfigurere Application

For å konfigurere hvordan kubernetes-manifester skal deployes må det opprettes en kobling mellom manifestene og et namespace. Dette gjøres ved å konfigurere en `Application`.

1. I git-repoet med manifestene, opprett en fil under

`<basepath>/applications/<miljø>/<komponent>.yaml`. Basepath er rotmappen definert i tenant-definisjon som inneholder `application` og `applicationsets` mappene som ArgoCD vil lete etter.

2. Kopier malen og fyll inn verdier:

Helm

```
kind: Application
apiVersion: argoproj.io/v1alpha1
metadata:
  name: <tenant>-<applikasjon>-<miljø> ❶
  namespace: gitops-developers
spec:
  project: <virksomhet>-<tenant> ❷
  source:
    repoURL: <git-repo> ❸
    targetRevision: HEAD
    path: charts/<applikasjon> ❹
```

```

helm:
  valueFiles:
    - values.yaml
    - environment/<miljø>/values.yaml ⑤
destination:
  namespace: <tenant>-<miljø> ⑥
  server: https://kubernetes.default.svc
syncPolicy:
  automated:
    prune: true
    selfHeal: true
    allowEmpty: false

```

- ① Navn på Application -ressursen. Må være unik for hele clusteret. Vi anbefaler derfor navnekonvensjonen <tenant>-<applikasjon>-<miljø> .
- ② Navn på tenant.
- ③ URL til git-repo med manifestene. For eksempel <https://github.com/oslokommune-uke/uke-dokumentasjon-argocd>.
- ④ Path til Helm chartet i git-repoet.
- ⑤ Spesifiser values.yaml fil med verdier som skal overskrives i miljøet.
- ⑥ Navn på namespace som skal deployes til.

Kustomize

```

kind: Application
apiVersion: argoproj.io/v1alpha1
metadata:
  name: <tenant>-<applikasjon>-<miljø> ①
  namespace: gitops-developers
spec:
  project: <virksomhet>-<tenant> ②
  source:
    repoURL: <git-repo> ③
    targetRevision: HEAD
    path: <path/to/manifests/> ④
  destination:
    namespace: <tenant>-<miljø> ⑤
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: false

```

- a. Navn på Application -ressursen. Må være unik for hele clusteret. Vi anbefaler derfor navnekonvensjonen <tenant>-<applikasjon>-<miljø> .
- b. Navn på tenant.
- c. URL til git-repo med manifestene. For eksempel <https://github.com/oslokommune-uke/uke-dokumentasjon-argocd>.
- d. Path til kubernetes-manifestene i git-repoet. Spesifiser mappen med kustomization.yaml i miljøet som skal deployes til.
- e. Navn på namespace som skal deployes til.

3. Merge Argo CD-manifestet til main.

✓ Argo CD vil deploye manifestene som refereres til i source .

Referanse

Konfigurering av Argo CD Application vil forklares videre. Se eventuelt [Application Specification](#) for offisiell referanse.

Source

Konfigurerer kilden til kubernetes-manifestene.

For Helm refererer vi til path for chartet, `values.yaml` med generelle verdier, og en miljø-spesifikk `values.yaml`.

```
source:
  repoURL: https://github.com/oslokommune-uke/uke-guestbook-argocd
  targetRevision: HEAD
  path: charts/guestbook
  helm:
    valueFiles:
      - values.yaml
      - environment/<miljø>/values.yaml
```

For Kustomize refererer vi til mappen hvor `kustomization.yaml` ligger:

```
source:
  repoURL: https://github.com/oslokommune-uke/uke-guestbook-argocd
  targetRevision: HEAD
  path: manifests/guestbook/envs/prod
```

For mer referanse om konfigurering av source, se Argo CD sine brukerguides om [Helm](#) og [Kustomize](#).

Target Revision

Feltet `targetRevision` brukes til å sette hvilken "versjon" av Git-repoet som skal brukes som kilden til sannhet for manifestene. I prod bruker man vanligvis `HEAD` (siste commit i main), men det kan settes til andre verdier dersom en ønsker å bruke en Git tag, en branch eller en spesifikk commit som kilde.

Konfigurering av `targetRevision` til å være en Git tag lar deg for eksempel deploye manifeste til test-miljøer fra hvilken som helst commit på hvilken som helst branch. Dette gjør det mulig å deploye og teste endringer uten å merge endringene til main.

Navnekonvensjonen vi anbefaler å følge:

```
targetRevision: <komponentnavn>-<miljø>
```

For eksempel, ved å sette `targetRevision` til `guestbook-test` kan vi bruke taggen `guestbook-test` i git til å peke til branchen vi ønsker å deploye til test-miljøet. Argo CD Application vil se slik ut:

```
kind: Application
apiVersion: argoproj.io/v1alpha1
metadata:
  name: uke-demo-guestbook-test
  namespace: gitops-developers
spec:
  project: uke-demo
  source:
    repoURL: https://github.com/oslokommune-uke/uke-demo-argocd
    targetRevision: guestbook-test
    path: charts/guestbook
    helm:
      valueFiles:
        - values.yaml
        - environment/test/values.yaml
```

Når `targetRevision` er konfigurert kan dette brukes til å kontinuerlig deploye til test-miljøer med bruk av Git tags. Les mer om bruk av Target Revision i [Kontinuerlig deployment](#).

Autosync

Feltet `syncPolicy` brukes til å automatisk synkronisere tilstanden mellom git og hva som kjører i clusteret. Dette settes som oftest til `automated` slik at endringer i git blir deployet uten å trenge noen manuell håndtering.

Les mer om [Automated Sync Policy](#) og [Sync options](#) i den offisielle dokumentasjonen til Argo CD.

For eksempel, for å automatisk synkronisere tilstanden uten muligheten for å slette alle ressurser dersom manifestene blir borte:

```
syncPolicy:
  automated:
    selfHeal: true # Specifies if partial app sync should be executed when resources are
                    # changed only in target Kubernetes cluster and no git change detected ( false by default ).
    prune: false # Specifies if resources should be pruned during auto-syncing ( false by
                  # default ).
    allowEmpty: false # Allows deleting all application resources during automatic syncing
                      # ( false by default ).
```

Eksempel

For å deploye applikasjonen `guestbook` til prod kan en `Application` se slik ut:

Helm

```
kind: Application
apiVersion: argoproj.io/v1alpha1
metadata:
  name: uke-demo-guestbook-prod
  namespace: gitops-developers
spec:
  project: uke-demo
  source:
    repoURL: https://github.com/oslokommune-uke/uke-demo-argocd
    targetRevision: HEAD
    path: charts/guestbook
    helm:
      valueFiles:
        - values.yaml
        - environment/prod/values.yaml
  destination:
    namespace: uke-demo-prod
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      selfHeal: true
      prune: false
      allowEmpty: false
```

Kustomize

```
kind: Application
apiVersion: argoproj.io/v1alpha1
metadata:
  name: uke-demo-guestbook-prod
  namespace: gitops-developers
spec:
  project: uke-demo
  source:
```

```
repoURL: https://github.com/oslokommune-uke/uke-demo-argocd
targetRevision: HEAD
path: manifests/guestbook/envs/prod
destination:
  namespace: uke-demo-prod
  server: https://kubernetes.default.svc
syncPolicy:
  automated:
    selfHeal: true
    prune: false
    allowEmpty: false
```

Videre lesing

Nå som du har oversikt over `Application` kan du gå videre til å lese mer om hvordan `ApplicationSet` kan brukes for å unngå boiler-plate, se [Bruke ApplicationSet](#).

Sopra Steria sin OpenShift-dokumentasjon inneholder også informasjon om `Application` i [User-defined ArgoCD applications](#)